Implementation of a Clock-based Intrusion Detection System (CIDS) on CAN Bus Simulation Environment

Davide Baggio - 2119982
davide.baggio.1@studenti.unipd.it

Abstract

This report aims to reproduce the methodology described in "Fingerprinting Electronic Control Units for Vehicle Intrusion Detection" [1]. The paper presents the Clock-based Intrusion Detection System (CIDS) algorithm, which is a new type of IDS capable of detecting fabrication, suspension, and masquerade attacks. In this report, these attacks have been replicated in a simulated environment using ICSim and demonstrate the robustness of fingerprinting ECUs via message intervals by leveraging Recursive Least Squares (RLS) for clock skew estimation and Cumulative Sum (CUSUM) for anomaly detection.

1 Introduction

The Controller Area Network (CAN) bus is the standard for intra-vehicle communication, however its broadcast nature makes it susceptible to spoofing attacks. CIDS identifies ECUs based on their unique hardware clock signatures, allowing for effective detection of anomalies in message timing.

2 System Setup

The experiments were conducted on a Linux Ubuntu 25.10 virtual machine using Instrument Cluster Simulator (ICSim) [2] to emulate a CAN bus environment without need for physical hardware. It includes a dashboard `./icsim` and a control panel `./controls` to interact in the environment and observe the effects of attacks in real-time.

- **Target ECU**: Blinkers (ID 0x188) with a nominal period of 20ms.
- **Interface**: Virtual CAN (`vcan0`).
- **Logic**: Python scripts utilizing the `can` library.

To reproduce the attacks the user has to clone this project repository [3] and also the ICSim one [2]. Then `cd ICSim/` and move there the following files:

- `ECU20ms.py`,
- `attack_fabrication.py`,
- `attack_suspension.py`,
- `attack_masquerade.py`,
- `plot_cids.py`,
- `run.sh`

At this point the user can simply run the bash script which automates the entire process: baseline collection, attack execution, post-attack monitoring and CIDS analysis. The generated log file is stored in the `Dumps` folder while the CIDS analysis plot is saved in the `CIDS` folder.

Table 1: CIDS and RLS Hyperparameters

Parameter	Value	Description
T_{ref}	0.020 s	Nominal period of the target ECU (ID 0x188).
λ	0.9995	Forgetting factor for RLS adaptation.
σ (std_err)	0.0001	Normalization factor for identification error.
κ	0.001	Sensitivity offset to filter out bus noise.
Γ (Threshold)	5.0	Cumulative limit to trigger an intrusion alert.
Baseline Limit	1500 msgs	Training window (approx. 30s) before parameter freeze.

3 Methodology

3.1 CIDS Algorithm

CIDS operates on the principle that every ECU has a unique "clock skew" (S), a persistent deviation from the nominal frequency caused by hardware manufacturing tolerances. By measuring the arrival intervals of periodic messages (e.g., the 20ms period of ID 0x188), CIDS constructs a timing baseline. An intrusion is detected when the observed timing of messages deviates significantly from the established fingerprint. The Recursive Least Squares (RLS) estimates the skew and covariance trying to minimize a weighted linear least squares cost function relating to the input signals.

3.2 Attack Scenarios

As mentioned before, the attacks are the following:

1. **Fabrication:** High-frequency injection (1ms) to override commands.

2. **Suspension:** DoS using ID 0x000 (highest priority in the CAN protocol) to silence the target ECU.

3. **Masquerade:** Spoofing the 20ms period but with an attacker-specific clock skew.

4 Experimental Results

The system behavior was analyzed across three phases:

1. **Baseline:** Normal operation of the ECU for 30 seconds to establish the clock skew fingerprint.

2. **Attack:** Execution of one of the three attacks for 20 seconds.

3. **Recovery:** Post-attack monitoring for 20 seconds to observe system recovery.

The effectiveness of the detection depends on the calibration of the RLS and CUSUM parameters. Based on the simulation environment, the values shown in Table 1 were selected to ensure a high detection rate while minimizing false positives:

4.0.1 Fabrication Attack

Identification error spikes during injection as visible in Figure 1. CIDS identifies the ECU as compromised because the L^+ value exceeds the threshold ($\Gamma = 5$) almost immediately after the attack begins.

4.0.2 Suspension Attack (DoS)

This attack causes "stuttering" in the error plot due to bus saturation jitter as visible in Figure 2. The massive use of messages with ID 0x000 silences the target ECU, impeding the correct arbitration of 0x188 messages and causing the L^+ values to slightly vary because of the large

quantity of messages sent with the same ID, clogging the network traffic.

4.0.3 Masquerade Attack

This is the stealthiest attack among the three, as it respects message frequency. However, as visible in Figure 3, CIDS detects the intrusion through shifts in clock skew, as the L^+ values start to grow after the attack implementation.

5 Conclusion

The implementation confirms that CIDS is effective in a simulated environment. The freeze logic successfully prevents the model from adapting to the attacker's clock, ensuring detection.

References

[1] K. T. Cho and K. G. Shin, "Fingerprinting electronic control units for vehicle intrusion detection," in *Proceedings of the 25th USENIX Conference on Security Symposium (SEC'16)*, Austin, TX, USA: USENIX Association, 2016, pp. 911–927.

[2] ICSim GitHub Repository https://github.com/zombieCraig/ICSim.

[3] Project Repository https://github.com/dvdbaggio/CIDS

A CIDS Analysis Plots

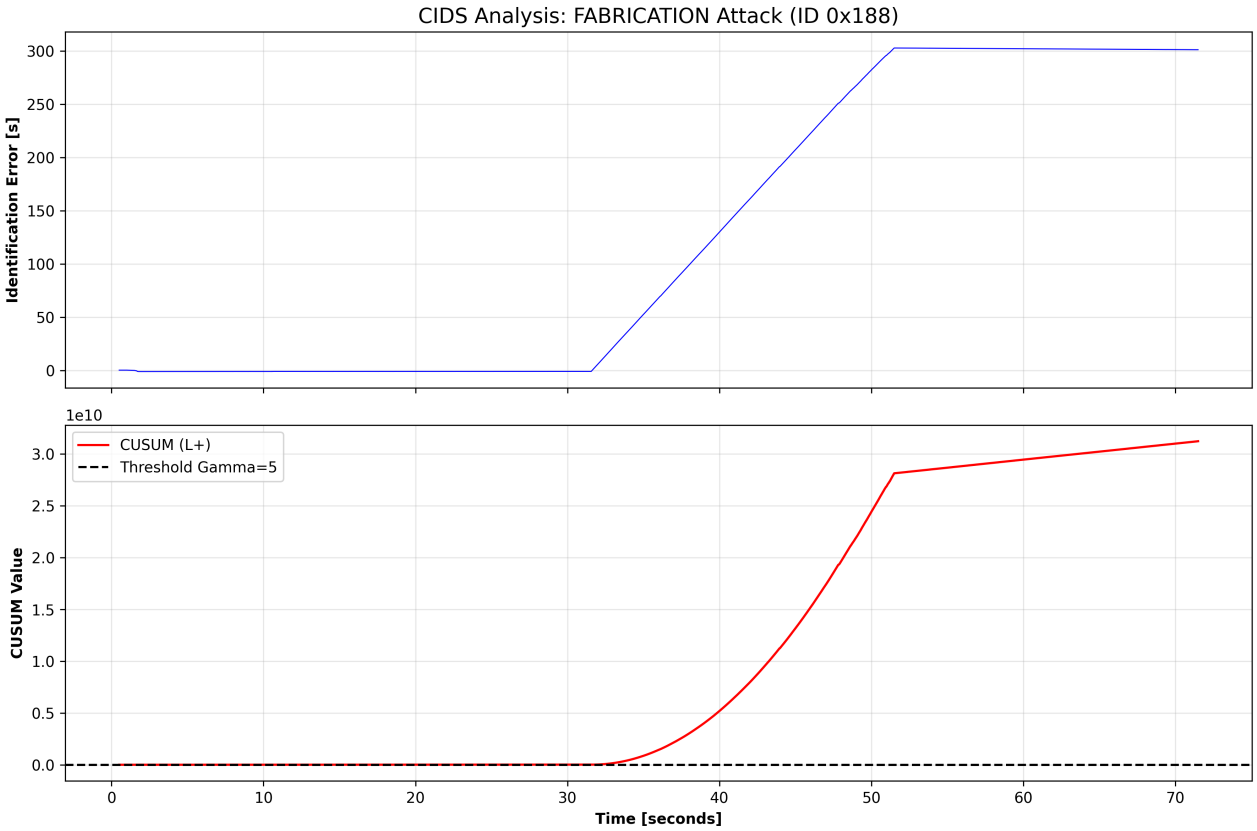

Figure 1: CIDS Analysis: Fabrication Attack. Note the exponential explosion of the CUSUM value.

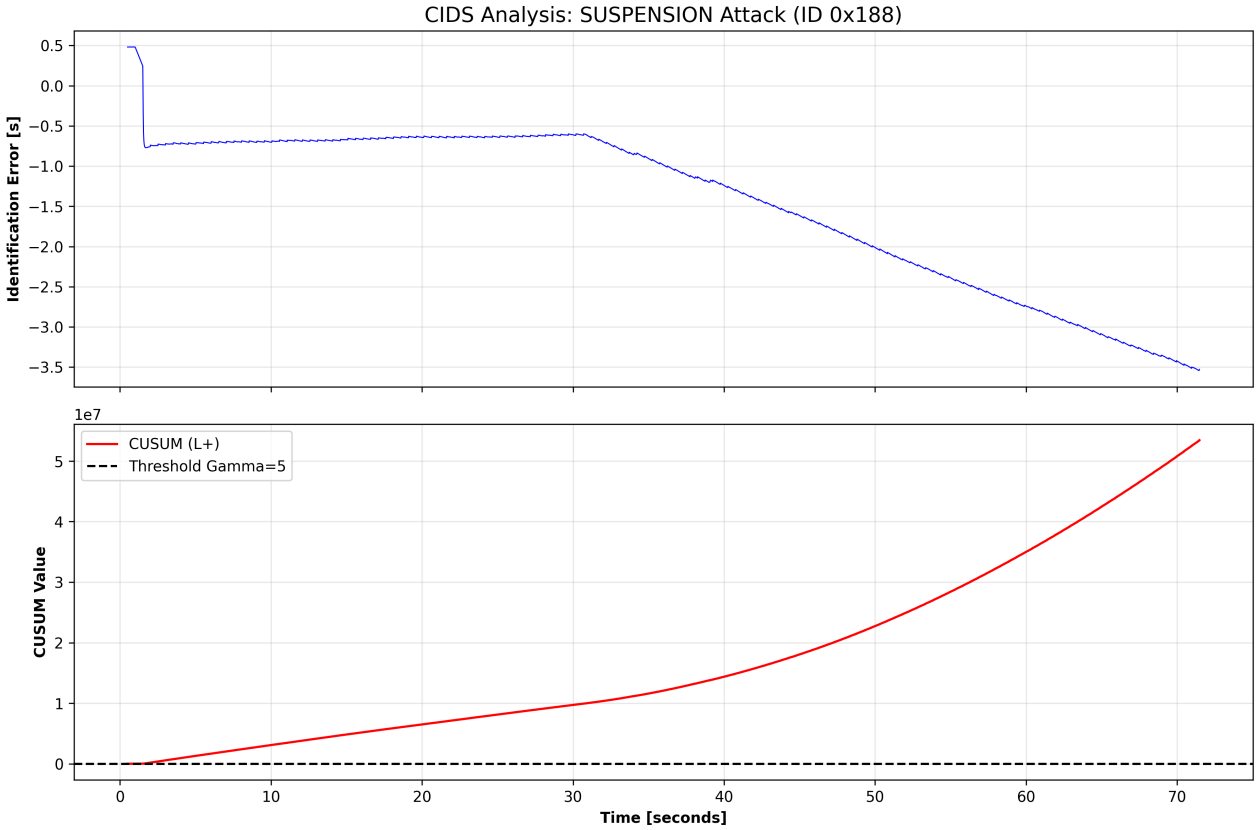

Figure 2: CIDS Analysis: Suspension Attack. Timing jitter caused by DoS results in logarithmic CUSUM growth.

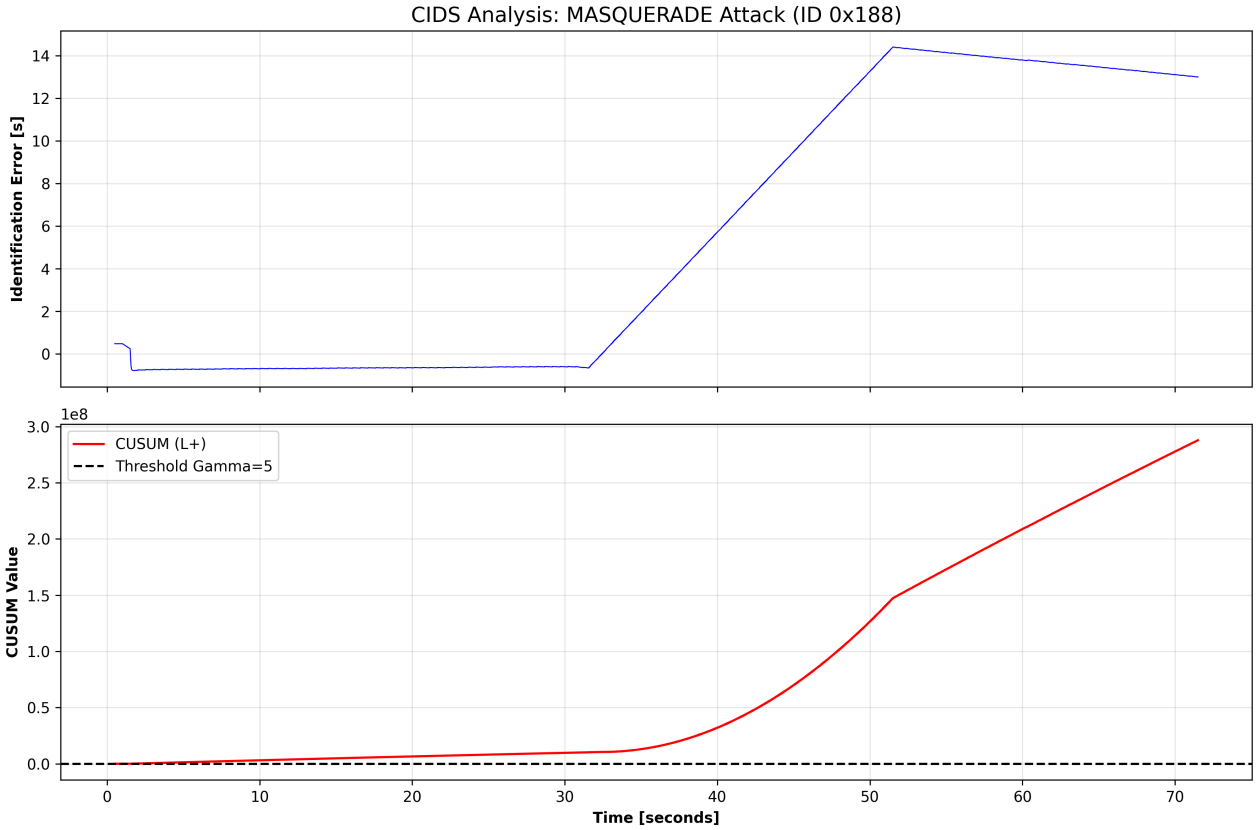

Figure 3: CIDS Analysis: Masquerade Attack. The linear drift identifies the hardware clock discrepancy.